MPI Transplants to Multi-Agent Systems, and Two of Its Rules Invert

AgentMPI Project

ABSTRACT

Multi-agent language-model systems occupy the position parallel computing occupied in 1991: every group writes its own coordination layer, none of the layers compose, and no portable interface exists to write a *harness* against. Parallel computing left that position by standardising the calls a program makes, and this paper shows that the same interface, MPI, transplants to agent coordination once its cost model is restated [34, 35]. We specify, implement and evaluate AgentMPI, a protocol for writing multi-agent systems rather than a multi-agent system.

Five asymmetries between an MPI process and a language-model executor change several of MPI's answers. The scarce resource is the receiver's context window rather than bandwidth, so MPI's eager limit becomes a token-denominated flow-control rule and MPI's safe-program discipline becomes a checkable property we call *context safety*. The reduction operator costs seconds rather than nanoseconds, so algorithm selection minimises operator applications on the critical path rather than message volume: at $p = 128$ the schedule MPI prefers for short messages applies the operator 896 times where reduce-then-broadcast applies it 127 times. A shared control plane replaces a point-to-point network, and MPI's barrier hierarchy collapses.

Agent-evaluated reductions expose a limitation we found in a run rather than predicted. A canonical reduction tree makes such a reduction reproducible but not consistent, because two branches can meet the same conflict and resolve it oppositely with no node positioned to notice. We name this the *locality of merge* and answer it with *conflict lifting*: the operator's codomain gains a conflict set whose join is a semilattice, so the root receives the same conflicts for every tree shape and arbitrates each exactly once.

AgentMPI ships as a specification, a reference runtime over a six-operation device interface with five transports, a conformance suite of 470 protocol and 195 device assertions run unchanged against every transport, and a process manager. Real agent populations exercised it: paired eight-rank translation runs, a hundred-rank run under executor oversubscription, and a production translation of a Russian biography into three languages by 16, 32 and 64 process ranks, then by 128 and 256 ranks over several machines. Three results are negative, and the multi-machine runs measured the transport's ceiling instead of a book; we report them as such.

KEYWORDS

multi-agent systems, message passing, MPI, collective communication, fault tolerance, LLM agents

1 INTRODUCTION

A multi-agent language-model system is written today the way a parallel program was written in 1991. The author picks a coordination pattern, a group chat, a supervisor tree or a directed graph of handoffs, and implements it from scratch against whatever agent kit is at hand. The result composes with nobody else's work, cannot move to a different agent host without a rewrite, and fails in ways every other group rediscovers on its own.

Parallel computing left that state by agreeing on an interface [34, 44]. The MPI Forum standardised neither a runtime nor a language nor a programming model; it standardised the calls a program makes to communicate, and left the process manager, the algorithms and the transport to implementations. That decision is why a program written in 1996 still runs, and why a dozen vendors shipped MPI-1 within two years of the document.

This paper asks whether the same move is available now and answers that it mostly is. AgentMPI is a protocol a harness is written with, in the sense that MPI is a library a parallel program is written with. It decides neither how many agents to run, nor what they do, nor how to prompt them, nor how to recover; it supplies the mechanisms with which those decisions are expressed. It is semantics-thin, a message being an opaque body plus an envelope, which rejects the KQML and FIPA-ACL lineage, whose mentalistic semantics proved unverifiable, in favour of MPI's stance: standardise the mechanism and leave meaning to the application.

The transplant is not mechanical. Five properties of a language-model executor differ from those of an MPI process (Section 3), and each forces a deviation. Two of MPI's algorithm-selection rules invert. One of MPI's cost terms, the reduction operator, goes from free to dominant. One of MPI's non-goals, fault tolerance, becomes the common case. One resource with no MPI analogue, the receiver's context window, turns algorithm selection from an optimisation into a feasibility test.

Contributions.

(1) A specification (Section 4) that maps MPI's chapters onto agent coordination and states, for each mechanism, whether it transfers, transfers with a changed unit, or does not transfer. It is written in the standard's own idiom: normative text with explicit rationale.
(2) The locality of merge (Section 5), a limitation of agent-evaluated reductions observed in a real run, together with two mechanisms, conflict lifting and invariant verification, and a shape-independence property for the first.

(3) A cost model and re-derived selection rules (Section 6) in which the operator term dominates, with the resulting inversions stated and measured.
(4) A reference implementation organised around a narrow waist (Section 7): six device operations, five transports including one that spans machines through a git remote, one conformance suite run against all of them, and a process manager that makes a rank an operating-system process on one machine or across several.
(5) An evaluation with real agent populations (Section 8), from eight-rank paired runs to a production translation by 64 process ranks, and the multi-machine runs whose failures measured two transport limits.

2 BACKGROUND: WHAT MPI STANDARDISED

MPI succeeded by standardising existing practice under strict rules, and three of its decisions carry directly into this work. The section first recalls the Forum's rules and non-goals, then states the three decisions: communicators, safe programs, and performance portability.

MPI was assembled in 1992–1994 from a field of incompatible message-passing libraries, among them p4, PARMACS, Chameleon, Zipcode, PVM, Express and NX, each of which solved part of the problem and none of which was portable [14, 24, 34, 44]. The Forum's rules were restrictive and effective: standardise existing practice rather than invent, and decline everything not yet understood. MPI-1's non-goals were as consequential as its goals. It is not a language, and it provides neither shared memory, nor debugging, nor I/O, nor threads, nor task management, nor fault tolerance. Declining those kept it small enough to implement.

Communicators. A communicator pairs a group with a private context that partitions the message namespace. Pre-MPI libraries addressed messages by destination and tag, so a library's messages could be intercepted by its caller whenever two integers collided. MPI's designers identify the context as the reason MPI could support libraries at all [23, 27, 41]. The agent-world version of that collision is not hypothetical: a reviewer's critique intended for the author of module A and consumed by the author of module B is the same bug, and the shared group chat makes it certain rather than possible.

Safe programs. MPI refuses to guarantee buffering. A program that completes only because an implementation happened to buffer is called unsafe, and the standard's advice is to test a program by replacing every standard-mode send with a synchronous one. The test is mechanical and rarely run, because the penalty for skipping it is a deadlock on someone else's machine.

Performance portability. The standard specifies semantics, and implementations choose algorithms. The Hockney model $T = \alpha + \beta n$ and its successors let an implementation pick a collective algorithm by message size and process count [3, 13, 38, 42]. Evidence on which parts of MPI applications use motivates our conformance levels [32].

Table 1: The five asymmetries that drive every deviation.

	MPI process	AgentMPI executor
A1 Determinism	deterministic	may produce a different, plausible, wrong result
A2 Scarce resource	bandwidth	**context window**
A3 Operator cost	nanoseconds; free	seconds to minutes; dominant
A4 Latency	tight	heavy-tailed; max of p samples
A5 Failure	fail-stop, rare, fatal	frequent, partial, sometimes silent

3 WHY AN AGENT IS NOT A PROCESS

Five asymmetries between an MPI process and a language-model executor drive every deviation in the specification, and a sixth runs the other way. Table 1 lists the five; the paragraphs below trace each to the section it shapes, and the last paragraph states the sixth.

Their consequences run through the rest of the paper. A2 forces the eager threshold to be denominated in tokens (Section 4.2). A3 inverts algorithm selection (Section 6). A4 motivates quorum collectives. A5 requires the whole fault-tolerance chapter. A1 requires reduction reproducibility and, separately, consistency to be declared properties (Section 5).

One asymmetry runs the other way and is easy to miss. In MPI the control plane is a point-to-point network. In AgentMPI it can be, and in our implementation is, a shared medium every rank can read. That structural difference collapses MPI's barrier hierarchy: a counting barrier costs $2(p-1)$ operations against dissemination's $p\lceil \log_2 p \rceil$, 510 against 2,048 at $p = 256$, so it is cheaper at every size, and it is also the only algorithm that can name the ranks that did not arrive. MPI avoids counting barriers at scale for a reason that does not apply here.

4 DESIGN

The specification keeps MPI's mechanisms and changes their units, their identification, and their failure behaviour. This section summarises it; the normative document accompanies the artifact. Section 4.1 separates a rank from its executor, Section 4.2 restates flow control in tokens, Section 4.3 restates datatypes as views and contracts, Section 4.4 changes how collectives are identified, Section 4.5 gives shared state MPI-3's one-sided discipline, and Section 4.6 adopts ULFM's fault-tolerance stance.

4.1 Ranks, Epochs, and Identity

A rank is a durable role rather than a process: a number, a mailbox, a context budget and an accumulated ledger. Its physical embodiment is an executor bound to it for a bounded interval, identified by an epoch. In MPI a rank *is* a process, and the identification is harmless because MPI processes live for the whole job; an agent session does not, so tying rank identity to session identity would make every timeout a communicator rebuild.

The epoch is a fencing token. Leases alone do not suffice: an executor cannot know its lease expired mid-step, so between expiry and its next call two executors believe they own the rank. A monotone token checked on every operation closes that window and makes a zombie harmless rather than corrupting.

Identity is ambient but must be assertable, a correction a run forced on us. Ambient identity eliminated the error it was designed to eliminate, agents passing the wrong rank, and replaced it with a worse one: agents silently *being* the wrong rank, because on one host shell sessions were shared between concurrent agents and the rank variable was overwritten between calls. The fix keeps ambient identity and makes the assertion cheap.

4.2 Flow Control in Tokens

Every rank has a context budget, and delivering a body charges it. Below an eager threshold of 700 tokens by default, a body is pushed into the receiver; above it, the receiver gets an envelope and a content-addressed handle and materialises explicitly. This is MPI's eager limit with the unit changed from bytes to tokens, and it exists for the same reason: below the limit, pushing unsolicited is cheaper than a handshake; above it, the receiver's buffer, here its attention, is too precious to fill without permission. Long contexts cost even when they fit, because model accuracy degrades with position and length and serving systems treat the window as a managed resource [29, 33]. Both are reasons to account for the window in the protocol rather than in the prompt.

Two things do not transfer. First, in MPI eager and rendezvous are invisible because both deliver the same bytes; here they differ in what arrives, so they must be visible. Second, exceeding the buffer does not deadlock; it silently degrades the receiver's output.

The second point makes MPI's safe-program discipline more important here, so the runtime mechanises it. A program is *context-safe* if it completes for every assignment of unexpected-message budgets, however small. The reference implementation runs a harness's communication skeleton under zero-buffer semantics, reports the ranks that wedge and any cycle among them, and re-runs with every send declared rendezvous so it can say whether the harness is repairable by transport choice alone. A ring in which every rank sends before it receives, the shape agent harnesses write by default, is diagnosed with its cycle named and the repair stated.

4.3 Views and Contracts

An MPI derived datatype is a declarative description of how to access a buffer, and its value is that the access pattern is data the library can optimise rather than control flow it cannot see. The scarce resource here is context, so the analogous question is not which bytes but which tokens, and how few. A *view* is a deterministic, model-free projection: `head:n`, `grep:pat`, `keys:a,b`, `outline`, `stat`. Semantic compression is deliberately not a view, because it costs an operator application the harness must be able to see.

Contracts transplant the second job of MPI datatypes: a type signature checked at both ends, which turns silent corruption into a loud error. A contract may also require a payload to identify itself, which turns a misrouted scatter slice into an error at the receiver rather than a plausible wrong answer three phases later.

4.4 Collectives

Collectives are labelled rather than positional. MPI identifies the kth collective by program order, which is safe when the caller is a compiled program. An executor's program order is not: it retries, skips and reorders. Positional identification would pair one rank's second call with everyone else's first and return a confidently wrong result. Labels cannot make a reordered schedule complete, but they make the mismatch named.

Gather returns a manifest rather than a concatenation. At $p = 128$ with 4,000-token contributions, an inlining allgather charges one rank 508,000 tokens and moves 65,024,000 in total; a handle-based one charges 5,080 and moves 650,240, a factor of 100 in peak residency. Concatenation is not a reasonable default at any interesting p.

Barriers carry a policy, one of wait, raise, proceed, shrink and revoke, because only the harness author knows whether a missing participant degrades the result or kills it. Collectives may carry a quorum, which releases without closing: a straggler still passes through, because a quorum that closed would guarantee that precisely the slowest ranks fail.

4.5 Windows

Every multi-agent system grows a shared scratchpad, and every one gets the concurrency wrong the same way: two agents read it, both edit a private copy, and the second write discards the first's work. MPI-3's one-sided chapter supplies both the vocabulary and the tools. `put` is the blind overwrite that loses work; `accumulate` combines atomically, so "union this finding into the shared findings" needs no lock; `compare_and_swap` is how work is claimed, and unlike a lock it cannot be held by a dead executor.

Two departures follow from the executor's nature. Locks are leased and carry a fencing token, because an executor can wander off where an MPI process cannot. Enumeration is free while reading is not: listing a window's keys with their sizes and writers costs a few tokens per key, which is what makes a blackboard usable by an executor with a bounded context.

4.6 Fault Tolerance

The specification adopts ULFM's stance wholesale: expose failure and provide mechanisms rather than policy [5, 6]. `revoke` is the non-obvious primitive. When a rank fails, the survivors are the problem, blocked inside collectives that can never complete, and revocation makes them all fail fast so they reach the recovery path together. `shrink` derives a communicator over an agreed survivor set and also offers FT-MPI's in-place mode, because renumbering invalidates a rank-to-work mapping that for agents is baked into prompts [16].

Recovery proceeds by briefing rather than checkpoint. No memory image exists to restore, so process checkpointing has no analogue; durable-execution replay does. The dominant failure here is a confident wrong answer, the exact analogue of silent data corruption, and HPC's answer to that is not checkpoint and restart, which preserves the corruption, but algorithm-based fault tolerance [8, 28]. A replacement receives the answers to five questions it cannot guess: what was I assigned, what did I publish, what did I receive, what did I promise that is outstanding, and what did I record for myself.

Detection is lazy, local and two-phase. It is an eventually perfect detector in Chandra and Toueg's classification, and the leases carry fencing tokens for the reason Gray and Cheriton give [12, 20]. A single-phase lease detector convicted 1091 times in twenty minutes on a real host, because a thinking executor and a dead one look identical. A blocked rank must renew its own lease while waiting; omitting that once made a barrier convict everyone who arrived early.

5 THE LOCALITY OF MERGE

An agent-evaluated reduction can be reproducible and still contradict itself, because each merge sees two operands and a global invariant sees the whole leaf multiset. This section states the failure as observed, generalises it, gives the mechanism that answers it and the property that mechanism has, and closes with the half the mechanism cannot reach.

MPI requires a user reduction operator to be associative and reorders freely. The guarantee is deliberately weak, since floating-point `MPI_SUM` is not bitwise reproducible across process counts, and practitioners accept it because rounding error bounds the discrepancy [2]. An operator implemented by a language model is not associative, is lossy, is expensive and is non-deterministic, and nothing bounds the discrepancy. AgentMPI therefore makes the algebra declared rather than assumed and refuses schedules the declaration does not license. That much is straightforward.

What is not straightforward is a failure a run showed us.

> In an eight-rank reduction over interface proposals, two branches of the tree independently encountered the *same* conflict, which module defines the shared exception type, and resolved it in *opposite* directions. Both resolutions were recorded, so the merged result contained two contradictory rulings under one identifier, and the tree had no way to notice: each merge saw a locally consistent pair. Two ranks then implemented different halves of the result. Separately, the same reduction dropped four of eight modules from one section of its output despite an explicit instruction never to drop a module.

A canonical tree shape makes a reduction reproducible. It does not make the reduction consistent, and conflating the two is the mistake.

The observation, stated generally. Let a reduction be a fold of a binary operator $\oplus$ over a tree T whose leaves are the ranks' contributions, and call a predicate φ a global invariant if its truth depends on the whole leaf multiset. An internal node of T sees exactly two operands. If two nodes u and v, neither an ancestor of the other, both encounter evidence bearing on φ, no node of T can reconcile them: their lowest common ancestor receives u's and v's outputs, and the information that would reveal the inconsistency, what each decided and why, is not in $\oplus$'s codomain. Enlarging the tree cannot help. Only enlarging the codomain can.

Conflict lifting. Let the operator's codomain be $V \times C$, where C is a set of undecided conflicts, and require that an operator which cannot decide a contested item from its two operands alone lift the item into C rather than decide it. Let C's join be set union.

Proposition 5.1 (Shape independence). *The conflict set arriving at the root of a fold over $\oplus$ is the same for every tree shape over the same leaf multiset, and no contested item is decided more than once.*

The argument takes two lines. The conflict component's join is associative, commutative and idempotent, so it is a semilattice fold, and a semilattice fold is independent of the order and shape of the fold. The value component never decides a contested item, so it cannot decide one twice. Arbitration at the root therefore decides each conflict exactly once, over the same set, whatever the schedule. The suite verifies this exhaustively: over every binary fold order at $p \le 6$, all Catalan$(p-1)$ shapes, and over random orders to $p = 32$, the conflict set is invariant, while a control operator that decides locally is not.

Why it is cheap. Lifting buys the one-ruling-per-key invariant for one extra operator application and $O(|C|)$ tokens. The alternative that also buys it, a serial chain in which the accumulator carries every prior decision, costs $p-1$ critical-path applications against a tree's $\lceil \log_2 p \rceil$: at $p = 64$, 63 against 6.

Implementation warning. Represent the lifted state as a map from key to the set of distinct values seen, and derive the value and conflict presentation from it. The obvious implementation, which lifts on first disagreement and removes the key from the value, is not idempotent: a third contributor finds the key absent and reinstates it, so the conflict set depends on fold order after all. We shipped that version, and its own shape-independence test caught it.

Table 2: Total operator applications, allreduce. Recursive doubling and reduce-then-broadcast put the same number on the critical path.

p	8	16	32	64	128
recursive doubling	24	64	160	384	896
reduce + broadcast	7	15	31	63	127
ratio	3.4	4.3	5.2	6.1	7.1

The other half. "Never drop a module" is not a conflict any pair of operands exhibits, since every local merge is individually faithful, so lifting is blind to it. It is a property of the leaf multiset and the result, so an operator may declare a global invariant that is checked once, after the reduction closes, and reported with the missing items named. Neither mechanism makes an agent operator correct. They make two specific, observed, expensive failure modes into loud errors, which is the most a protocol can claim.

6 COST MODEL AND ALGORITHM SELECTION

When the operator term dominates, two of MPI's selection rules invert and a third question, admissibility, precedes optimisation. This section states the model, fits its constants, and derives the three consequences in turn.

The model separates time from price. An operation moving n tokens costs

$$T = \alpha + \beta n + \gamma k + \lambda, \tag{1}$$

$$C = c_{\text{tok}}\, n + c_{\text{op}}\, K, \tag{2}$$

where γ is the cost of one operator application by an executor, k the number of applications on the critical path, K the number in total, and λ the executor's own heavy-tailed think time. Equation (1) is Hockney's model with two terms added; Equation (2) is new, because in MPI no one pays per operator application.

Measured on the reference implementation (Section 8.1), $\alpha = 0.730$ ms and $\beta = 0.480\,\mu$s per token on the SQLite transport, which gives a half-bandwidth point of 1,521 tokens. That point lies above the size of a typical agent artifact, so latency-optimal algorithms win over a wider range of sizes than in MPI. Against them, γ for a language-model merge is seconds to minutes. At $p = 128$ the entire protocol cost of an allreduce is 15.09 ms, which is 0.215 % of the total against a one-second operator and 0.0072 % against a thirty-second one.

Runtime operators want no tree. When the implementation can apply the operator itself, as with set union, JSON merge or max, a shared control plane folds every contribution in place: one round, zero messages. A tree adds rounds and buys nothing. This is the in-network-aggregation regime, and it is the common case for the operators harnesses should use. A sweep over γ confirms it: at $\gamma = 0$ the winner is `flat`, and at $\gamma = 30\,$s it is `reduce_bcast`, with the flat schedule 10 × behind (1890.0 s against 180.0 s).

Agent operators want MPI's tree and reject MPI's best tree. A binomial tree puts $\lceil \log_2 p \rceil$ applications on the critical path against a chain's $p-1$; the catalogue also includes Bruck's algorithm and Rabenseifner's, and the scan schedules follow Ladner and Fischer and Blelloch [7, 9, 31, 39]. Recursive-doubling allreduce, MPI's standard choice for short messages precisely because redundant arithmetic is free, applies the operator $p\lceil \log_2 p \rceil$ times in total for the same critical path (Table 2) [38, 42]. The algorithm that wins for bytes loses for agents, and it loses on the price axis, which is why the model reports time and price separately.

Admissibility precedes optimisation. An algorithm whose peak per-rank residency exceeds a context budget is infeasible rather than slow. At $p = 128$ with 4,000-token payloads, every inlining allgather in the catalogue is rejected with its peak residency named; the handle-based variants are admissible. MPI has no analogue: there, every algorithm runs.

7 IMPLEMENTATION

The implementation separates portable semantics from transport at a six-operation waist, checks the separation with one conformance suite, and adds the two things the specification leaves to implementations, a command binding and a process manager. Section 7.1 describes the waist and its transports, Section 7.2 the suite and what it found, Section 7.3 the binding, and Section 7.4 the process manager and the transport that spans machines.

7.1 The Narrow Waist

MPICH's portability comes from an abstract device interface separating portable MPI semantics from transport, and Open MPI makes the same move with its component frameworks [10, 21, 22]. AgentMPI copies the structure for a different reason: an interface with one implementation is a library, and what makes this a standard is that the semantics can be stated and checked independently of any transport.

Six operations sit at the waist: `append` (durable, totally ordered), `match` (atomic first-fit claim), `scan`, `cas`, `lease` and `clock`. Everything above them, communicators, matching, collectives, windows, the ledger, revoke, shrink and agree, is written in terms of those six. Fewer would not do: `append` and `scan` suffice for a single writer, but a receive must not be handed to two ranks, and that atomicity cannot be recovered above the device. More are not needed: window history is a `scan` over versions, the failure detector is a `scan` over leases, and the deadlock detector is a `scan` over blocked receives.

Five transports ship and share no code below the interface: SQLite in WAL mode, a filesystem journal using one file per record with an advisory lock, an in-process dictionary, a git device whose compare-and-swap cell is a ref on a shared remote, and a daemon in front of that git device (Section 7.4). 11,633 lines of implementation sit above the waist.

7.2 The Conformance Suite

195 device assertions and 470 protocol assertions, each naming the specification section it enforces, run unchanged against

every transport. The suite uses only public interfaces, so pointing it at a different implementation means changing one fixture.

The suite earns its keep. In its first pass it found six defects, of which two matter beyond this codebase. An administrative kill was retractable by its victim's next heartbeat, which would have made every fault-injection measurement meaningless. SQLite's type affinity returned integers as strings for four indexed fields, so a wildcard receive posted as -1 read back as `"-1"` and silently stopped matching, a divergence that existed on one transport and would otherwise have surfaced as an inexplicable protocol bug months later. A suite parametrised over every transport exists for exactly that divergence.

7.3 The Command Binding

For a language-model executor the binding is a command-line tool, because that is the only interface an agent reliably has to a stateful library: it cannot hold a handle across turns, cannot link a shared object, and its function calls are invocations whose output lands in its context window. Eight properties are normative, each because its absence cost a run: assertable identity; an identity echo on every call; errors that carry a concrete next action and a retryable flag; internal retry; sizes in tokens; a free path to disk on every operation that hands back a payload; the manual as a command; and print only commands that exist. The last has a test that walks every emitted command string through the real parser, because an early reduction directive printed a subcommand spelled with a space where the real one is hyphenated, and roughly ten agents reported it, several while peers were blocked behind them.

7.4 Ranks as Processes, and a Job Across Machines

MPI leaves the process manager unspecified, and so does AgentMPI; the reference implementation supplies one, `ampirun`, because the experiments needed it. It starts one operating-system process per rank with the rank's identity in its environment, block-distributes the ranks over the machines of a job, supervises the processes, and restarts a dead one through the runtime's `respawn` so the successor takes a new epoch and re-enters its collectives. Each collective the successor already joined returns its stored result, traced as replayed with its wait measured from re-entry, and each task it already finished is read back from a window, so the successor resumes at the first thing its predecessor left unfinished. A machine that was recycled under a running job can rejoin it: the manager re-enters the job from the shared device and respawns the ranks the peers convicted in its absence.

Across machines, the ranks share nothing but a git remote. The git device treats a ref as its compare-and-swap cell (fetch, apply, commit, push, retry on rejection), and its mutations are pure functions of the device state so a rejected push re-applies them against fresher state. One push per mutation does not scale: thirty-two ranks on thirty-two machines made 1131 pushes and suffered 13415 rejections in forty-two minutes for four collectives. The daemon, `gitd`, owns the working tree on each machine and group-commits every mutation its ranks issue within a window into one push, the intra-node aggregation every production MPI performs before it touches the interconnect. The guarantee each rank observes is unchanged, and the pushes on the wire are bounded by the machine's rate of bursts rather than its ranks' rate of operations. Two rules of the daemon came from runs and are now normative: its readers take no lock when a writer fetched within the read interval, and its batch window widens while pushes are rejected (Section 8.7).

8 EVALUATION

Real agent populations ran against the protocol at every scale the account allowed, and the informative results are the ones that failed. The microbenchmarks (Section 8.1) measure the protocol alone so the agent runs can attribute their time. E1 (Section 8.2) is a paired translation experiment whose quality result is null. E2 (Section 8.3) measures the fault-tolerance claims with scripted ranks. Section 8.4 and Section 8.5 report what a session-capped host and shared shell sessions did to a hundred-rank run. E7 (Section 8.6) is the production translation by process ranks, on one machine to completion and across machines to the transport's limits.

8.1 Microbenchmarks

The protocol's own cost is milliseconds, and its per-token cost is a property of the layer above the waist rather than of any transport. No model is involved, deliberately: the job is to measure the protocol so the agent experiments can attribute their time correctly.

Fitting $T = \alpha + \beta n$ by ping-pong regression over payloads from 1 to 12,800 tokens gave $\alpha = 0.730$ ms on SQLite, 3.947 ms on the filesystem journal, and 0.096 ms in memory. The half-bandwidth point is 1,521 tokens on SQLite and 8,104 on the journal. One result was not anticipated: β agrees to within 1.5 % across the three single-machine transports. The per-token cost is serialisation and token counting above the waist, not transport below it. Only α is a property of the device, which is a useful fact for the cost model and an argument that the waist sits in the right place.

8.2 E1: Translating a Book

The glossary collective found the disagreement it was built to find, and the disagreement did not reach the artifact. The paragraphs below give the task, the harness, the scoring, the mechanism's behaviour, the null result, and the threats.

The task is native translation of a book, chosen because it is almost embarrassingly parallel. Each passage translates independently; what does not is the rendering of names and coined terms recurring across passages. Ignore the coupling and the Tin Woodman acquires four names; serialise to avoid it and the harness pays p executor latencies.

We split *The Wonderful Wizard of Oz* into 100 passages and extracted 21 recurring terms by capitalisation frequency,

Table 3: E1, two paired arms at two sizes. Both metrics are null at both sizes, and the collective costs wall clock at both.

	$p = 8$		$p = 32$	
	gloss.	ctrl	gloss.	ctrl
Consistency	1.000	1.000	0.994	0.994
strict	0.9474	0.9474	0.9509	0.9632
Terms scored	10	10	15	15
Wall clock (s)	400	109	346	255
Result tokens	6,648	6,082	26,480	24,406

filtered by a model-free rule under which a candidate that also appears lower-cased anywhere in the corpus is dropped. The harness has six phases, each one collective: scatter the passages; an agent phase proposing renderings; an allreduce with `union`, which lifts disagreements rather than deciding them; arbitration at the root; a broadcast of the binding glossary; an agent phase translating; a gather. The protocol lives in the harness rather than the prompt. An executor's entire obligation is to read a prompt file, write a result file and submit; the worker bootstrap prompt mentions no communicator, collective, barrier or glossary. That is the design claim stated as an artifact, and it is why the same experiment runs against a deterministic function or a hundred live agents without changing a line.

Scoring is mechanical. For each recurring term, the scorer finds which rendering each passage's translation used and takes the modal rendering's share, weighted by how many passages contain the term. The candidate renderings searched for are pooled across all arms compared, which matters: our first scorer built the vocabulary from each run's own term sheets, and since only the treatment arm produces term sheets, the control was scored against an almost-empty vocabulary and flattered by it.

The mechanism worked and scaled as predicted. At $p = 8$ the reduction agreed 17 terms unanimously and lifted 3 as conflicts; at $p = 32$ it agreed 8 and lifted 12. Quadrupling the population quadrupled the disagreement it found, which is what the conflict machinery exists to surface. The root arbitrated each conflict once at both sizes. Every one of the 12 conflicts was of one kind, whether the Spanish definite article belongs in the rendering, *el Espantapájaros* against *Espantapájaros*; not one was a disagreement about the noun.

The result is null at both sizes. Table 3 gives the numbers. Weighted terminology consistency is 1.000 in all four arms; under the strict metric that counts the article as part of the decision, the control arm at $p = 32$ scores higher than the treatment (0.9632 against 0.9509). The glossary cost 3.7 × the control's wall clock at $p = 8$ and 1.4 × at $p = 32$.

We had predicted that the null at $p = 8$ was a scale effect and that a larger population would surface terms with weaker priors. The prediction was half right and instructive in the half that was wrong. The population's stated conventions diverged more at scale, with four times as many lifted conflicts, exactly as predicted. That divergence did not reach the artifact, because the translators disagreed about the definite article, and in running prose the grammar of the sentence determines the article. Each translator wrote *el Espantapájaros* or *Espantapájaros* as the sentence required, whichever it had declared. The collective correctly identified a real disagreement in what the population said it would do, and that disagreement did not matter for what the population produced. This mirrors the caution of Section 5: agreement is not correctness, and here disagreement was not incorrectness either. The operational lesson for a harness author is one we did not expect to report: before paying a collective to remove a coupling, measure whether the coupling costs anything.

One threat runs the other way. An executor in the treatment arm reported, unprompted, that it capitalised compass words mid-sentence because the glossary lists them capitalised and it reasoned the check would be mechanical; it chose a less natural rendering to match what it believed the scorer looked for. That is Goodhart's law inside the experiment, and it biases the treatment arm upward on the metric we report (79% against the control's 70% on mid-sentence capitalisation). The treatment did not beat the control even so, which makes the null a floor rather than a point estimate. A metric an executor can see is a metric an executor will aim at. The other threats are the usual ones: $n = 1$ per cell, one model family, one language pair, and one corpus whose recurring terms are famous enough that a strong model has seen them. A task whose terminology is novel, an internal codebase or a new API, is where we would expect the collective to pay, and we have not run it.

8.3 E2: Fault Tolerance

Every fault-tolerance claim the specification makes held when ranks died at the worst moments, and the last claim held exactly and no further. The question here is not how a model behaves under failure, which E1 and the conformance suite cover, but whether the recovery machinery does what Section 4 says. That question has a definite answer, and getting it means killing ranks at controlled points many times, which is what one should not pay a language model to sit through. Scripted ranks at $p = 12$ ran five scenarios, each a specification claim stated as a measurement, and all 5 of 5 held.

The last row of Table 4 repays careful reading. With one rank dying mid-claim, 0 tasks were done twice and 1 was left orphaned, claimed by a dead executor and never finished. That is exactly what the protocol promises and no more. Compare-and-swap guarantees a task is not done twice; finishing a dead claimant's task requires a supervisor, and reporting the orphan count is how a harness author learns how much they must still do themselves.

Table 4: E2: each row is a claim from the specification, measured.

Claim	Measured
A rank killed before a collective must not hang it	3 killed, 9 contributors, omission recorded
Revocation frees a survivor blocked *inside* a collective	freed after 1.32 s with `ERR_REVOKED`
Concurrent shrinks converge	11 ranks shrinking, 1 communicator
A replacement can resume from the briefing alone	5/5 published cells, plus the open collective and the memo
Claimed work is never done twice	24 tasks, 23 done, 0 duplicated

8.4 Oversubscription, and a Launcher Constraint

A host that caps concurrent sessions leaves a job with ranks nobody will start, and the protocol's two answers are oversubscription and the join deadline. Our agent host caps concurrent sessions at ten. This is no accident of our setup; every host we have used imposes some such cap, and it is exactly the condition that leaves ranks unserved. The $p = 32$ runs above were served by 8 executors at oversubscription four.

Oversubscription has a precise MPI analogue: `mpirun -np 100` on eight cores runs a hundred ranks on eight cores. It works here for the same reason it works there, because a rank is a durable role whose state lives outside its executor, and it is available only because the protocol lives in the harness. An executor serving ten ranks agent-side would have to be inside ten barriers at once and would deadlock immediately; here the harness thread blocks in the collective and the executor blocks on nothing.

The join deadline answers the rank that never starts. A rank's lease starts when the rank is requested rather than when its executor first calls in. Without that rule, a rank whose executor never starts has no lease to expire, so it is neither alive nor failed and every peer waits for it forever. With it, a launcher that can start six of twenty-two ranks produces sixteen detectable no-shows.

8.5 Two Identities, and the One We Thought Was Checked

An identity assertion made of the same material as the identity it asserts cannot catch a corruption that rewrites both, and a run showed us that ours did not. Two identities travel with every task in these experiments. The protocol identity is the rank an operation acts as, which the binding lets a caller assert with `--expect-rank`. The provenance label is an environment variable added so that a scale claim would have per-executor evidence; it is outside the protocol and nothing checks it.

On a host where shell sessions are shared between concurrently running agents, both drifted. Across 199 tasks and 19 executors the provenance label was wrong on 18 of them (9.0 %). In every run with one executor per rank, and in the $p = 32$ runs at oversubscription four, it drifted 0 times.

We first reported that the protocol identity held because it was asserted. That was wrong, and two executors told us so. One wrote: "the `next` call carried `--expect-rank 3` and still returned rank 2's identity." Another diagnosed it exactly: "the guard didn't fire because the clobber replaced `--rank` and `--expect-rank` *together*." Both are right, for two independent reasons, and the reasons are more useful than the claim we had made.

The implementation defect. The assertion was wired into the library API and into the ordinary CLI operations, and not into the `worker` subcommand, the only surface any executor in these experiments touched. The conformance suite covered the paths we had thought about. What caught the one misroute that was caught was a weaker mechanism, the broker's check that a submitted task belongs to a rank the caller serves. That check is a serve-set membership test rather than an identity assertion, and it fires only at submit, after the work is done.

The design defect. An assertion compares two values the caller supplies. A corruption that rewrites both the acting rank and the assertion leaves them consistent, so the check passes and the operation acts as somebody else. No amount of asserting fixes this, because the assertion is made of the same material as the thing it asserts about. The mechanism that does fix it was already in the specification and already implemented: a per-rank launch token, issued by the launcher and recorded in the job, which no accident of the environment can make self-consistent. We had the mechanism and not the property, because the path that needed it did not call it.

What the executors did instead. Both misrouted claims were handled correctly by agent judgement rather than by the protocol. One executor abandoned its task with an accurate reason. The other declined even to abandon it, reasoning that `give-up` would also have acted as the wrong rank and would have discarded another executor's work, and let the claim lease-expire instead, which the broker requeued and its rightful server completed. That is a better decision than the protocol was positioned to enforce, and we would rather not rely on it. All four defects are fixed and regression-tested: identity is asserted on the worker path, the emitted `submit` and `give-up` commands carry `--expect-rank`, and the identity flags are accepted on either side of the subcommand.

8.6 E7: The Production Translation by Process Ranks

A population of process ranks finished the book at every single-machine scale, and adding ranks bought little wall time because the run ends when its slowest model ends. The paragraphs below give what changed from E1's design, the executor, the model pool the provider forced, the method, and the results on one machine; Section 8.7 gives the runs across machines.

E3, the session-rank predecessor of this experiment, wrote the protocol for the task and could not staff it: its executors were agent-host sessions, the host capped concurrent sessions at ten, and every run above $p = 16$ spent its wall time waiting for an executor to exist. E7 keeps that design, the same collectives for the same reasons, and changes what a rank is. A rank is an operating-system process. Its executor is a chat-completions call with a bounded tool loop. `ampirun` starts it on one machine or across several. Executor supply stops being the limiting resource, so for the first time the population scales against a fixed workload: the same 2013 Russian biography, at 4 scales from $p = 16$ to $p = 128$ on one machine and at $p = 128$ and $p = 256$ across machines.

What changed, and why. Four things distinguish E7 from E1, and each is a protocol mechanism rather than a prompt. The unit is the paragraph: a page-cut book has 95 prose pages and a paragraph-cut one has 1232, so p can reach 256 with two to five paragraphs a rank, segments stay contiguous and character-balanced, and the seam exchange keeps its meaning. Arbitration is agent-evaluated and parallel: after the `allreduce` every rank holds the same conflict set, so rank r arbitrates batch r of it, the rulings are gathered to the root, and the root commits them with `op_arbitrate` exactly once, which spreads over the population the serial step that would otherwise grow with it. Results are checkpointed and collectives replay: every agent result is written to a window before the rank proceeds, so a rank whose process dies is restarted with a new epoch and resumes at the first thing its predecessor did not finish, and `--die-fraction` makes the cost of that lifecycle failure a measured quantity. Shared state sits under a lock: a translator who settles a term the glossary did not cover records it in a per-chapter ledger under an exclusive, leased, fenced window lock, first writer wins, because a term already present with a different rendering is a clash to record rather than a value to overwrite.

The executor. A raw endpoint has no body. The executor (`ampitools/model.py`) gives it the smallest one that suffices: an encyclopaedia search, an article, a page fetch, called through the provider's function-calling schema in a loop bounded both by steps and by cumulative prompt tokens. Contract violations are repaired in the same conversation rather than by a fresh attempt. Every call's exact prompt and completion tokens, tool calls and cost land in the trace as `task.*` events, which the analysis reads as it reads the broker's claim and submit pair. Two defects the executor taught us are fixed and traced. A tool loop whose tools are silently withdrawn produces empty replies, and withdrawing them in the model's own turn does not. A rate limit is not a failure of a task: the first attempt on one model lost six ranks to a twenty-requests-a-minute cap because 429s were retried like transport faults, and a rank waiting on a rate limit is now a rank blocked.

The population is heterogeneous by necessity. The provider caps every capable model at twenty requests a minute per model for the account these runs used, so a population of sixty-four ranks on one model would queue on the cap rather than translate. Rank r therefore runs model $r \bmod 8$ of a pool of eight at every scale, which multiplies the aggregate limit by eight and makes the executors heterogeneous in a way the protocol never asks about. A segment's model is a function of its rank alone, the mix is the same across the series, and the trace records the model behind every task, so per-model latency and cost are a by-product of the run rather than an extra experiment.

Method. Every run cut the same book by the same rule and used the same prompts, the same research cap (48 terms) and budget (3 per rank), the same model pool, and the same policies (quorum 1, barrier policy *proceed*, one restart per rank). What changed with p was the segment size and the number of segments, which is strong scaling. The launch plan named every rank requested before any ran; each node's launch record carried the machine's kernel boot id; each rank's report carried its epoch and the model it ran on; and the trace carried everything else. The source is in copyright and never left the untracked working directory; the repository carries the evidence, the population's glossary, its research findings with their sources, and its amendment ledger.

The population finished, and the protocol never wedged. Every run in Table 5 closed all of its collectives and assembled a book. At $p = 16$ the sixteen processes rendered 98.5% of the 1232 paragraphs in 51.4 minutes for \$7.61; at $p = 64$, 96.0% in 37.9 minutes for \$11.06. The missing coverage is not the protocol's: it belongs to the segments of ranks whose model degenerated (below), which the runtime dropped from their collectives after the restart budget was spent so that the rest of the population could close.

More ranks bought little wall time, and the trace says why. Wall time fell from 51.4 to 39.9 to 37.9 minutes as p quadrupled, while executor work stayed near 4.2–6.6 rank-hours and blocked time grew from 9.3 to 32.7 rank-hours: the coordination share rose from 68.0% to 81.0%, and parallel efficiency fell from 30.6% to 16.2%. This is not protocol overhead, since on one machine the median collective released its slowest participant within a second. It is Amdahl's law with a heterogeneous denominator: the run ends when its last rank ends, and the model a rank drew decides which rank is last. Across the three runs the median translation chunk took 28 s on the fastest model in the pool and 119 s on the slowest, a 4.3× spread (Figure 1), and the longest single wait in every run was a seam exchange or the final spend reduction blocked on one rank whose model was slow or had been restarted. With a fixed book, adding ranks shortens every rank's segment but does not shorten the slowest model's proportion of it.

What the collectives did. The census lifted 22, 53 and 104 conflicts at the three scales, renderings on which ranks reading different parts of the book disagreed, and a model arbitrated every one exactly once, in parallel batches, before the research agenda was built from them. The 48 researched terms cite 121 sources at $p = 16$, and exactly one rank researched each term at every scale: the compare-and-swap claim on the research window turned 64 potential investigations of the Singer House

p	nodes	wall (min)	tasks	restarts	coord. share	efficiency
16	1	51.4	241	3	68.0%	30.6%
32	1	39.9	280	2	73.1%	25.3%
64	1	37.9	366	20	81.0%	16.2%
128	4	137.2	479	0	34.4%	6.4%

Table 5: E7 across single-machine scales. Coordination share is blocked rank-seconds over all rank-seconds; efficiency is achieved parallelism over p; coverage is the fraction of the 1232 paragraphs rendered into all three languages.

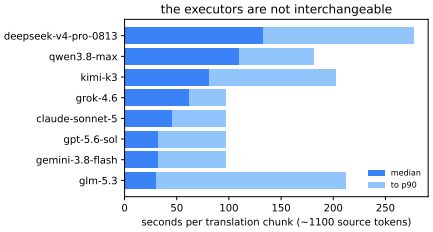

Figure 1: Seconds per translation chunk (about 1100 source tokens into three languages) by model, pooled over the single-machine runs: median, and median to 90th percentile. Rank r ran model r mod 8; the run ends when the slowest of them does.

into one. The glossary that bound the translation grew from 380 to 1404 terms with p, because more ranks survey more finely. The amendment ledger, written under the per-chapter lock, recorded 360, 276 and 207 terms the translators settled themselves, with 2, 1 and 4 clashes in which two ranks settled the same term differently in the same chapter, which is exactly the number the glossary phase had failed to prevent and which the ledger exists to make visible.

Failures were contained, and they were instructive. `ampirun` restarted 3, 2 and 20 ranks at the three scales; each successor took a new epoch, re-entered its closed collectives (0 collective re-entries traced as replays across the series), read back its finished tasks, and resumed. Most restarts at $p = 64$ had one cause, a race in the harness that delivered broadcast bodies to a file shared between ranks, found and fixed while the run was in progress; a rank restarted after the fix ran the fixed code, which is a property of process ranks that session ranks do not have. The failures that cost coverage had a different cause: one model in the pool degenerated into thousands of blank lines mid-object on the same paragraphs three times running, and in-conversation repair only reproduced the degeneration. A homogeneous population has no answer to that; a heterogeneous one does, and the executor now falls back to a second model from a fresh conversation when the first exhausts its attempts. The multi-machine runs carried that fix; the single-machine runs above did not.

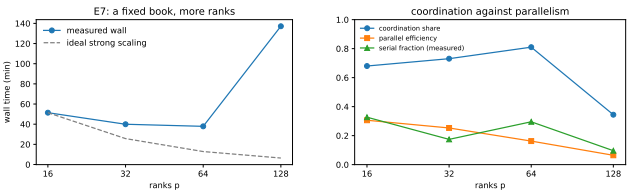

Figure 2: Left: wall time against p for a fixed book, with ideal strong scaling. Right: coordination share, parallel efficiency, and the measured serial fraction.

8.7 E7 Across Machines: What a Shared-Ref Transport Costs and Survives

Six attempts to run 128 ranks over four machines preceded the one that finished the book, and each of the five that did not measured something the completed run could not. The paragraphs below give the setup, what each attempt found, the completed run's numbers, and the recovery that followed when every session hosting it was frozen at once.

Each attempt was one job whose ranks were processes on several cloud machines sharing nothing but a git remote, over `gitd` (Section 7.4). With stub executors, eight ranks over two machines and a hosted remote had made 1371 operations in 441 pushes, and the median collective's slowest participant waited a minute against a second on one machine. That minute is the price of sharing nothing but a git remote, and the question was whether it matters against an executor whose one step costs minutes. It does not; what matters is what the ranks do while they wait.

The first attempt found a defect in the daemon. At 128 ranks over four machines, all 128 ranks surveyed their segments and the census closed, and 57 live ranks blocked in one broadcast were then convicted for silence. The daemon's readers waited on the lock its writer held for the whole push contest with the other machines, and a lock is not a queue: with thirty polling readers and a writer that reacquired the lock the moment it released it, one reader could wait longer than its lease. The trace records the anatomy exactly, suspicions with `silent_for` between 811 and 1457 s, convictions for `lease_expired`, and no retraction, because a rank that cannot complete a read cannot renew the lease that would keep it alive. The fix, readers take no lock when a writer fetched within the read interval, is in the device and in the specification (S15.2).

The second attempt measured the transport's ceiling. At 256 ranks over eight machines on the fixed daemon, 224 ranks surveyed their segments in 36 minutes and the census never closed. One compare-and-swap ref admits about one commit per round trip for the whole population, and eight daemons contending for it landed one push in ten: node 0's daemon made 45 pushes against 442 rejections in 80 minutes. A rank whose lease renewal is itself a push could not renew it, and 116 ranks whose last operation was joining the census were

convicted with a median silence of 252 s at suspicion. The ceiling is a property of the ref, and a machine's share of it falls with the number of machines. Four machines sit inside it; every later attempt used four.

The third and fourth attempts lost the machines, then the root's context. At 128 ranks over four machines no rank was convicted that was not dead. In the third, the machine hosting node 0 was recycled under the running population; its 32 ranks were convicted one by one, the process manager rejoined the job from the branch and respawned them with fresh epochs while the other 96 waited, and the population continued with the successors participating, until the account's usage limit froze every session at 123 minutes. The fourth reached the binding glossary and broadcast a poisoned one: the root's context ledger was near exhaustion when the glossary reduction returned, so the value the runtime handed it was a degraded view, canonical JSON with an elision marker in the middle of a string, and with no conflicts to arbitrate the root broadcast that string; 95 ranks died parsing it. The harness now reads a reduction's stored body by its handle when the ledger degraded the delivered one, and releases its ledger before every reduction whose result it must hold whole.

The fifth attempt found that every read was a write. Every phase up to the research fence completed. Then nothing happened for forty-six minutes: no model call in flight, no failure anywhere, the provider's dashboard flat. `py-spy` on any node-0 rank showed the same stack, a window read inside `charge` inside a compare-and-swap. The runtime charged the context ledger on every delivery, as the specification requires, and persisted the charge by writing the rank row, so a rank reading the forty-eight findings paid forty-eight group commits, a dozen seconds each at four machines, all thirty-two ranks of a node in step and none of them renewing a lease while blocked inside its own write. Nodes 1–3 reached the glossary reduction after sixteen minutes; node 0's ranks were at finding 45 of 48 when their 1800-second leases lapsed. A ledger charge is now local until the rank row is next written for another reason, and a read is never a device mutation (S6.1); the same run's trace showed the research tools failing on Cyrillic URLs and on Wikipedia's rate limit, both fixed.

The sixth attempt finished the book. On the corrected runtime, 128 ranks over 4 machines went from launch to the final reduction in 137.2 minutes: the four nodes joined within twelve minutes, 128 surveys took seventeen, the census reduction and seven parallel arbitration batches took three, 48 research findings took twelve, and the interval from research fence to bound glossary that had cost forty-six minutes the day before took four. Translation took twenty-seven minutes and the seam exchange an hour, both bounded by the slowest model, as at every scale on one machine. The population completed 479 tasks with 52 in-conversation repairs, lifted 160 conflicts, rendered 99.2% of the 1232 paragraphs and spent $13.70. Coordination took 34.4% of rank-time and parallel efficiency was 6.4%, against 81.0% and 16.2% at $p = 64$ on one machine: the shared-ref transport's round trips are what the extra rank-hours went to, and the run's wall time still fell short of one machine's because the population was waiting for the same slowest model. The trace carries every model exchange as a `task.call` event, 650 of them with 252 tool calls; 21 exchanges ended with the provider's own `error` finish and 3 tool calls failed, none of them the failures of the fifth attempt.

Then every session was frozen, and the job came back. Two minutes short of the final reduction, with every task done and 126 of 128 ranks arrived at it, the account's usage limit stopped all four sessions at once; the containers were reclaimed and every rank process died. The job lived on the branch. An hour later each node ran the launcher with `--rejoin`: ranks whose rows still said `running` were resumed, ranks the peers had by then convicted were respawned with fresh epochs, and every returning rank replayed its program from the start, collectives returning their stored results and tasks whose results were in the window skipped. Nothing was re-translated and the spend did not move. The replays found three defects in four hours, each fixed and each in the runtime or the harness rather than the protocol. A replaying rank's ledger is charged for every replayed delivery, so thirty ranks reached the translate phase with a full ledger and died on a degraded amendment ledger; the harness now releases at the translate boundary and reads the ledger uncharged. A respawned successor inherited its predecessor's ledger, 184,000 tokens into a budget of 200,000, and the root died on its first sizeable delivery, twice; a successor now inherits the budget alone. And the device's history, one whole state file per commit and never packed, reached 14 GB and filled node 0's disk two operations before the root finalised; the daemon now packs the repository every two hundred pushes. The root finalised at the fourth try, 428 minutes after launch, having gathered the manifest from ranks that had finished five hours earlier. The series reports the run's numbers from the trace up to the freeze (`window.json` names the boundary) and the full wall alongside.

What a run across machines needs. The measured facts are that a machine hosting 64 rank processes is unremarkable, that four machines sit inside the transport's ceiling and eight outside it, that a population frozen whole can be brought back from the branch at the price of one round trip per replayed operation, about an hour for this program at four machines, and that the instrument that found every defect above was the trace read against a rank's stack. A completed 256-rank run wants four machines, a session that never idles, and an account not shared with another experiment.

9 WHAT ONLY REAL AGENTS EXPOSED

Several rules in the specification exist because a run failed, and they are the part of this work that could not have been derived from MPI. The list below gives each rule with the

failure that produced it; the first ten came from session ranks and the last three from process ranks.

(1) **Executors give up far earlier than instructed.** One told to retry up to twenty times gave up after two and stalled its reduction tree. A protocol that depends on an executor's persistence is not a protocol; the binding retries internally and the default is to keep waiting.
(2) **Blocking is not evidence of death.** A rank waiting inside a barrier made no runtime calls, so the detector convicted it for waiting, and every rank that arrived early was declared failed. A blocked rank must renew its own lease.
(3) **Program order is not a collective identifier.** Hence labels.
(4) **Ambient identity without assertion is unsafe.** Shared shell sessions made one rank accumulate nine initialisations, one from nearly every other agent.
(5) **Never truncate a reduction operand.** A result may be summarised because its consumer is a reader; an operand may not, because its consumer is a function. Agents handed JSON clipped mid-string invented recovery hacks, one prefix-matching the content-addressed store by hand.
(6) **Print only commands that exist**, including the bootstrap prompt, which is also a command the binding prints. Ours placed the identity flags before the subcommand, where a reader expects a global option and where our parser rejected them, and roughly thirty executors each lost their first call to `invalid choice`. Any guard left out of a printed command is a guard present only when the agent thinks of it: the emitted `submit` omitted `--expect-rank`, and exactly the executors who noticed had it.
(7) **Give every payload a free path to disk.** Agents that lacked one reached into the object store directly rather than pay to see what was already a file.
(8) **A rank that never starts must still be detectable.**
(9) **A conditional send with an unconditional receive is a deadlock.** The protocol can only bound the damage; the advice is to send unconditionally, possibly empty. The static checker catches it.
(10) **When a protocol omits a mechanism, its users build it worse.** Eight agents given no coordination phase rebuilt an allgather by hand within ninety seconds of one another, then built runtime interface discovery on top at roughly five times the detection code and twice the total lines for the same graded behaviour, with two of the eight introducing defects inside their own detection logic. AgentMPI/1.0 therefore specifies interface declaration and verification, because the arms showed they are complementary: declaration alone reproduces the inconsistency of Section 5, and verification alone pushes the whole cost into every consumer.
(11) **A rate limit is a blocked rank, not a failed task.** The first production attempt on one model lost six ranks to a twenty-requests-a-minute cap because the executor retried a 429 like a transport fault, six times and then never. A rank waiting on a rate limit is a rank blocked, exactly like one blocked in a collective, and the wait is now bounded by patience rather than by a count.
(12) **A lock is not a queue.** A daemon whose readers waited on the lock its writer held for a push contest starved one reader for an hour at four machines, and the rank behind it was convicted alive. Readers that hold a fresh copy take no lock.
(13) **A machine that hosts ranks must never be idle.** A cloud session's machine is reclaimed when the session is idle, and node 0 of a 128-rank job was reclaimed twice under a running population. The protocol's answer, convict, respawn, replay, worked when the machine came back; the operational rule is that the session keeps its turn alive for the whole run.

10 DISCUSSION AND LIMITATIONS

The evidence supports a narrow claim, and this section states its boundaries. The paragraphs below give what the evidence does not show, the statistical power of the runs, the scale reached, the strongest objection to the design, and the sense in which agreement is not correctness.

What the evidence does not show. It does not show that AgentMPI produces better answers than an ad-hoc harness. Our one controlled quality comparison is null (Section 8.2), and we report it as such. What the evidence shows is narrower and still worth having: MPI's mechanisms have well-defined agent analogues, several of MPI's answers change in stated and measurable ways, the resulting interface can be specified precisely enough to be conformance tested, and real agent populations run against it.

A running job was hot-patched twice. An executor in the scale run reported an `ImportError` from a module another session was in the middle of fixing. Appendix D of our own specification warns about exactly this, and the warning was inadequate: it said to pin the runtime version, and the version string had not changed, because what changed was the code. The runtime now records a content hash of its own source at job creation and logs a diagnostic when a later caller's differs, advisory rather than fatal, because a developer iterating should not be locked out and a two-hour run should not die over a reflowed comment, but recorded, because a run whose runtime changed halfway through should say so in its own journal.

Statistical power. Every agent experiment here is $n = 1$ per arm. Agent runs are expensive and not reproducible, which is also why tracing is unconditional and why we commit raw per-rank artifacts rather than aggregates.

Scale. The largest completed agent run with session ranks is $p = 32$ over 8 executors, and with process ranks it is $p = 128$ on one machine. The hundred-rank session run did not finish: the host's session cap forced several waves of executors, and an operator error later destroyed the run's journal. The pull queue handled the first problem well, since a fresh wave could be added mid-run against exactly the ranks still queued, with no harness change and no restart, and 199 tasks were published across 19 executors; the second problem was ours. The 128- and 256-rank process runs across machines did not finish either, for the reasons Section 8.7 measures. Microbenchmarks reach $p = 128$ with scripted ranks.

The strongest objection. MPI's SPMD model presupposes a static rank set and interchangeable workers, and agent systems are neither. The objection is partly right: the fixed world size is a real constraint, and heterogeneous roles fit awkwardly into a model where rank r's program is the same as rank s's. Our defence is that a rank is a role rather than a worker, that communicators already express heterogeneity by partitioning, and that the constraint buys the thing the ad-hoc alternatives lack, a fixed membership against which a collective can be defined at all. The heterogeneity E7 was forced into, eight models across one population, cost the protocol nothing and cost the run its scaling.

Agreement is not correctness. A protocol that makes agreement cheap makes it cheap to agree on something false. In our translation run a glossary bound a term to a rendering correct nearly everywhere, and a rank correctly obeying it produced a wrong sentence in the one passage with a different sense. A consistency metric scores a uniformly wrong decision as perfect.

11 RELATED WORK

The pieces of AgentMPI each have an ancestor, and the synthesis is the claim. The paragraphs below place the work against agent protocols, agent communication languages, agent frameworks, and distributed computing.

Agent protocols. MCP standardises client-server tool access, and A2A and ACP standardise agent discovery and task handoff [1, 37]. None provides a communicator, a collective, shared state with atomics, failure detection, or flow control; they are complementary rather than competing, and AgentMPI could run over any of them as a transport.

Agent communication languages. KQML and FIPA-ACL define speech acts with mentalistic semantics [17, 18, 30]. The critique that those semantics are unverifiable from outside applies with more force to a language model, and we take MPI's road instead.

Frameworks. AutoGen, LangGraph, CrewAI, MetaGPT and their relatives supply coordination patterns [45]. Empirical work on how such systems fail independently identifies several of the modes our failure model enumerates [11]. They are the analogue of the pre-MPI libraries: each solves part of the problem, and none composes with another. A framework makes the coordination decisions; a protocol provides the mechanisms with which to make them.

Distributed computing. `Win_fence` is Valiant's BSP superstep boundary [43]. Quorum collectives are stale-synchronous parallelism expressed as a collective [26]. Supervision with a bounded restart intensity is Erlang/OTP's [4, 15]. Recovery by briefing is durable execution [36]. Windows are a blackboard in the sense of HEARSAY-II and Linda with MPI-3's one-sided discipline imposed on it, and work claiming by compare-and-swap descends from the Contract Net [19, 25, 40]. We claim the synthesis rather than the parts.

12 CONCLUSION

MPI's designers solved a coordination problem under a particular cost model, and they were explicit about which of their decisions followed from that model. That explicitness makes the transplant possible: one can take the mechanism, change the cost model, and see which answers change. Most survive. Two selection rules invert, one hierarchy collapses, one non-goal becomes the common case, and one resource with no analogue turns optimisation into feasibility.

The part that did not come from MPI is the part we would most like to be wrong about. A reduction evaluated by agents can be perfectly reproducible and still internally contradictory, because merges are local and invariants are not. We can name the problem, give it a mechanism whose correctness is a two-line semilattice argument, and check that mechanism exhaustively. We cannot make an agent operator correct, and a protocol should not claim to.

The part that came from running it at scale is a set of limits with numbers on them. A population of process ranks finishes a book on one machine and pays for every extra rank in the coin of its slowest model. Across machines, the same population found the two limits of a transport built on a shared git ref, one a defect now fixed and one a ceiling now measured, and it found them by convicting its own live members. That is the failure detector working as specified on a device that was not, and the trace that recorded it is the artifact we would point a harness author to first.

REFERENCES

[1] A2A Project. 2026. Agent2Agent (A2A) Protocol Specification, version 1.0.0. Linux Foundation / Agentic AI Foundation. https://a2a-protocol.org/v1.0.0/specification/

[2] Peter Ahrens, James Demmel, and Hong Diep Nguyen. 2020. Algorithms for efficient reproducible floating point summation. *ACM Trans. Math. Software* 46, 3 (July 2020), 22:1–22:49. https://doi.org/10.1145/3389360

[3] Albert Alexandrov, Mihai F. Ionescu, Klaus E. Schauser, and Chris Scheiman. 1995. LogGP: Incorporating long messages into the LogP model — one step closer towards a realistic model for parallel computation. In *Proceedings of the Seventh Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA '95)*. ACM, 95–105. https://doi.org/10.1145/215399.215427

[4] Joe Armstrong. 2003. *Making Reliable Distributed Systems in the Presence of Software Errors.* Ph. D. Dissertation. Royal Institute of Technology (KTH), Stockholm, Sweden.

[5] Wesley Bland, Aurelien Bouteiller, Thomas Herault, George Bosilca, and Jack Dongarra. 2013. Post-Failure Recovery of MPI

Communication Capability: Design and Rationale. *The International Journal of High Performance Computing Applications* 27, 3 (2013), 244–254. https://doi.org/10.1177/1094342013488238

[6] Wesley Bland, Aurelien Bouteiller, Thomas Herault, Joshua Hursey, George Bosilca, and Jack J. Dongarra. 2012. An Evaluation of User-Level Failure Mitigation Support in MPI. In *Recent Advances in the Message Passing Interface (EuroMPI 2012) (Lecture Notes in Computer Science, Vol. 7490)*. Springer, 193–203. https://doi.org/10.1007/978-3-642-33518-1_24

[7] Guy E. Blelloch. 1990. *Prefix sums and their applications*. Technical Report CMU-CS-90-190. School of Computer Science, Carnegie Mellon University.

[8] George Bosilca, Rémi Delmas, Jack J. Dongarra, and Julien Langou. 2009. Algorithm-Based Fault Tolerance Applied to High Performance Computing. *J. Parallel and Distrib. Comput.* 69, 4 (2009), 410–416. https://doi.org/10.1016/j.jpdc.2008.12.002

[9] Jehoshua Bruck, Ching-Tien Ho, Shlomo Kipnis, Eli Upfal, and Derrick Weathersby. 1997. Efficient algorithms for all-to-all communications in multiport message-passing systems. *IEEE Transactions on Parallel and Distributed Systems* 8, 11 (Nov. 1997), 1143–1156. https://doi.org/10.1109/71.642949

[10] Darius Buntinas, Guillaume Mercier, and William Gropp. 2006. Design and Evaluation of Nemesis, a Scalable, Low-Latency, Message-Passing Communication Subsystem. In *Sixth IEEE International Symposium on Cluster Computing and the Grid (CCGrid '06)*. IEEE, Singapore, 521–530. https://doi.org/10.1109/CCGRID.2006.31

[11] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail?. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*. arXiv:2503.13657 https://arxiv.org/abs/2503.13657

[12] Tushar Deepak Chandra and Sam Toueg. 1996. Unreliable Failure Detectors for Reliable Distributed Systems. *J. ACM* 43, 2 (1996), 225–267. https://doi.org/10.1145/226643.226647

[13] David Culler, Richard Karp, David Patterson, Abhijit Sahay, Klaus Erik Schauser, Eunice Santos, Ramesh Subramonian, and Thorsten von Eicken. 1993. LogP: Towards a Realistic Model of Parallel Computation. In *Proceedings of the Fourth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '93)*. ACM, 1–12. https://doi.org/10.1145/155332.155333

[14] Jack J. Dongarra, Rolf Hempel, Anthony J. G. Hey, and David W. Walker. 1993. *A Proposal for a User-Level, Message-Passing Interface in a Distributed Memory Environment*. Technical Report ORNL/TM-12231. Oak Ridge National Laboratory.

[15] Ericsson AB. 2025. Supervisor Behaviour — Erlang/OTP System Documentation. https://www.erlang.org/docs/29/system/sup_princ.html.

[16] Graham E. Fagg and Jack J. Dongarra. 2000. FT-MPI: Fault Tolerant MPI, Supporting Dynamic Applications in a Dynamic World. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface (EuroPVM/MPI 2000) (Lecture Notes in Computer Science, Vol. 1908)*. Springer, 346–353. https://doi.org/10.1007/3-540-45255-9_47

[17] Tim Finin, Richard Fritzson, Don McKay, and Robin McEntire. 1994. KQML as an Agent Communication Language. In *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM '94)*. ACM, Gaithersburg, Maryland, USA, 456–463. https://doi.org/10.1145/191246.191322

[18] Foundation for Intelligent Physical Agents. 2002. *FIPA ACL Message Structure Specification*. Technical Report SC00061G. Foundation for Intelligent Physical Agents. http://www.fipa.org/specs/fipa00061/SC00061G.html

[19] David Gelernter. 1985. Generative Communication in Linda. *ACM Transactions on Programming Languages and Systems* 7, 1 (Jan. 1985), 80–112. https://doi.org/10.1145/2363.2433

[20] Cary G. Gray and David R. Cheriton. 1989. Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency. In *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles (SOSP '89)*. ACM, 202–210. https://doi.org/10.1145/74850.74870

[21] William Gropp and Ewing Lusk. 1994. *An Abstract Device Definition to Support the Implementation of a High-Level Point-to-Point Message-Passing Interface*. Technical Report Preprint MCS-P342-1193. Mathematics and Computer Science Division, Argonne National Laboratory, Argonne, IL, USA.

[22] William Gropp, Ewing Lusk, Nathan Doss, and Anthony Skjellum. 1996. A High-Performance, Portable Implementation of the MPI Message Passing Interface Standard. *Parallel Comput.* 22, 6 (1996), 789–828. https://doi.org/10.1016/0167-8191(96)00024-5

[23] William D. Gropp. 2001. Learning from the Success of MPI. In *High Performance Computing — HiPC 2001, 8th International Conference, Hyderabad, India, December 17–20, 2001 (Lecture Notes in Computer Science, Vol. 2228)*, Burkhard Monien, Viktor K. Prasanna, and Sriram Vajapeyam (Eds.). Springer, Berlin, Heidelberg, 81–92. https://doi.org/10.1007/3-540-45307-5_8

[24] William D. Gropp and Barry Smith. 1993. *Chameleon Parallel Programming Tools Users Manual*. Technical Report ANL-93/23. Argonne National Laboratory.

[25] Barbara Hayes-Roth. 1985. A Blackboard Architecture for Control. *Artificial Intelligence* 26, 3 (1985), 251–321. https://doi.org/10.1016/0004-3702(85)90063-3

[26] Qirong Ho, James Cipar, Henggang Cui, Jin Kyu Kim, Seunghak Lee, Phillip B. Gibbons, Garth A. Gibson, Gregory R. Ganger, and Eric P. Xing. 2013. More Effective Distributed ML via a Stale Synchronous Parallel Parameter Server. In *Advances in Neural Information Processing Systems 26 (NIPS 2013)*. 1223–1231.

[27] Torsten Hoefler and Marc Snir. 2011. Writing Parallel Libraries with MPI — Common Practice, Issues, and Extensions. In *Recent Advances in the Message Passing Interface, 18th European MPI Users' Group Meeting, EuroMPI 2011, Santorini, Greece, September 18–21, 2011 (Lecture Notes in Computer Science, Vol. 6960)*, Yiannis Cotronis, Anthony Danalis, Dimitrios S. Nikolopoulos, and Jack Dongarra (Eds.). Springer, Berlin, Heidelberg, 345–355. https://doi.org/10.1007/978-3-642-24449-0_45

[28] Kuang-Hua Huang and Jacob A. Abraham. 1984. Algorithm-Based Fault Tolerance for Matrix Operations. *IEEE Trans. Comput.* C-33, 6 (1984), 518–528. https://doi.org/10.1109/TC.1984.1676475

[29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*. ACM, 611–626. https://doi.org/10.1145/3600006.3613165

[30] Yannis Labrou and Tim Finin. 1994. A Semantics Approach for KQML — A General Purpose Communication Language for Software Agents. In *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM '94)*. ACM, Gaithersburg, Maryland, USA, 447–455. https://doi.org/10.1145/191246.191320

[31] Richard E. Ladner and Michael J. Fischer. 1980. Parallel prefix computation. *J. ACM* 27, 4 (Oct. 1980), 831–838. https://doi.org/10.1145/322217.322232

[32] Ignacio Laguna, Ryan Marshall, Kathryn Mohror, Martin Ruefenacht, Anthony Skjellum, and Nawrin Sultana. 2019. A Large-Scale Study of MPI Usage in Open-Source HPC Applications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '19)*. ACM, New York, NY, Article 31, 14 pages. https://doi.org/10.1145/3295500.3356176

[33] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. https://doi.org/10.1162/tacl_a_00638

[34] Message Passing Interface Forum. 1994. *MPI: A Message-Passing Interface Standard*. Technical Report CS-94-230. University of Tennessee, Knoxville.

[35] Message Passing Interface Forum. 2023. *MPI: A Message-Passing Interface Standard, Version 4.1*. Technical Report. University of Tennessee, Knoxville. https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf

[36] Microsoft. 2025. Durable Functions Overview. https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-overview.

[37] Model Context Protocol Contributors. 2026. Model Context Protocol Specification, revision 2026-07-28. Agentic AI Foundation / Linux Foundation. https://modelcontextprotocol.io/specification/2026-07-28/basic/index

[38] Rolf Rabenseifner. 2004. Optimization of collective reduction operations. In *Computational Science — ICCS 2004 (Lecture Notes in Computer Science, Vol. 3036)*. Springer, 1–9. https:

//doi.org/10.1007/978-3-540-24685-5_1

[39] Rolf Rabenseifner and Jesper Larsson Träff. 2004. More efficient reduction algorithms for non-power-of-two number of processors in message-passing parallel systems. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface (EuroPVM/MPI 2004) (Lecture Notes in Computer Science, Vol. 3241)*. Springer, 36–46. https://doi.org/10.1007/978-3-540-30218-6_13

[40] Reid G. Smith. 1980. The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver. *IEEE Trans. Comput.* C-29, 12 (Dec. 1980), 1104–1113. https://doi.org/10.1109/TC.1980.1675516

[41] Marc Snir, Steve Otto, Steven Huss-Lederman, David Walker, and Jack Dongarra. 1996. *MPI: The Complete Reference*. MIT Press, Cambridge, MA.

[42] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. 2005. Optimization of collective communication operations in MPICH. *The International Journal of High Performance Computing Applications* 19, 1 (Feb. 2005), 49–66. https://doi.org/10.1177/1094342005051521

[43] Leslie G. Valiant. 1990. A Bridging Model for Parallel Computation. *Commun. ACM* 33, 8 (Aug. 1990), 103–111. https://doi.org/10.1145/79173.79181

[44] David W. Walker. 1992. *Standards for Message-Passing in a Distributed Memory Environment*. Technical Report ORNL/TM-12147. Oak Ridge National Laboratory, Oak Ridge, TN.

[45] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI] https://arxiv.org/abs/2308.08155